

Getting Started with Python Testing Using Pytest

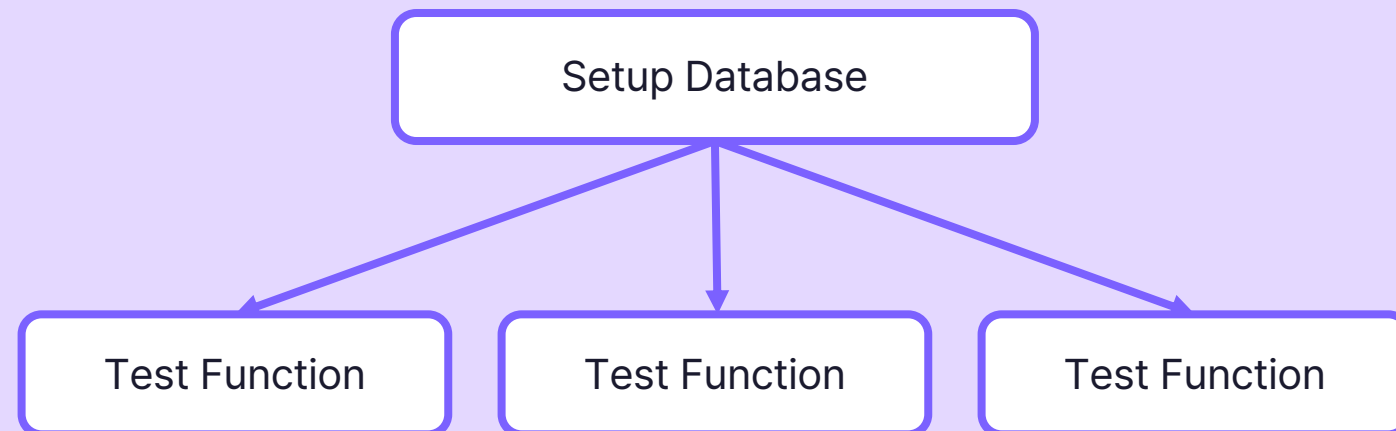
Pytest로 시작하는 파이썬 테스트

테스트 자동화하기

테스트 심화 – Fixture, Parametrize

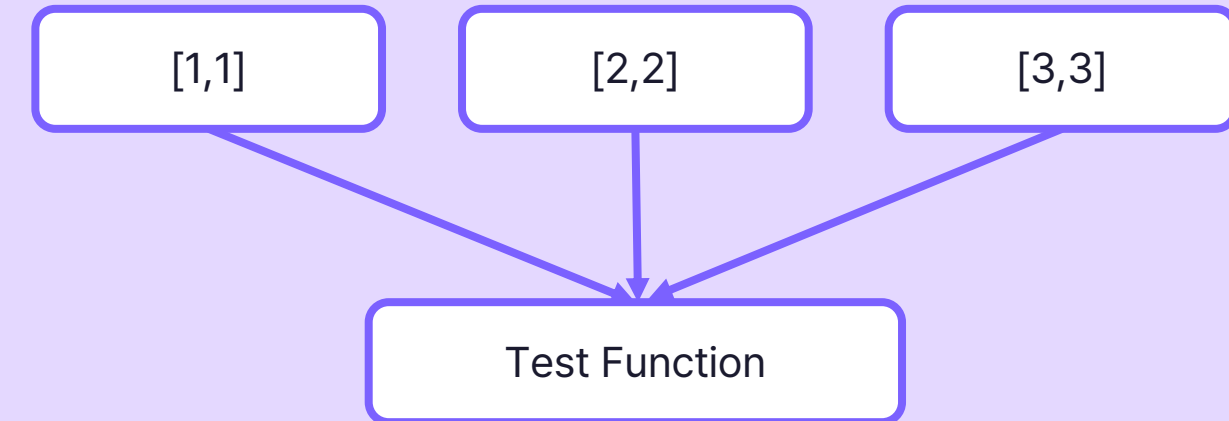
핵심 기능 : Fixture (테스트 환경의 효율적 통제)

Fixture - 사전 준비



- 테스트 전 필요한 준비 작업(DB 연결, 객체 생성 등)을 한 번만 정의
- conftest.py 파일에 작성 시, 모든 파일에서 import 없이 즉시 자동 공유 가능

Parametrize - 대량 검증



- `@pytest.mark.parametrize` 활용
- 하나의 테스트 함수에 여러 개의 '입력값-기대값' 세트를 주입하여 반복 코드 완벽 제거

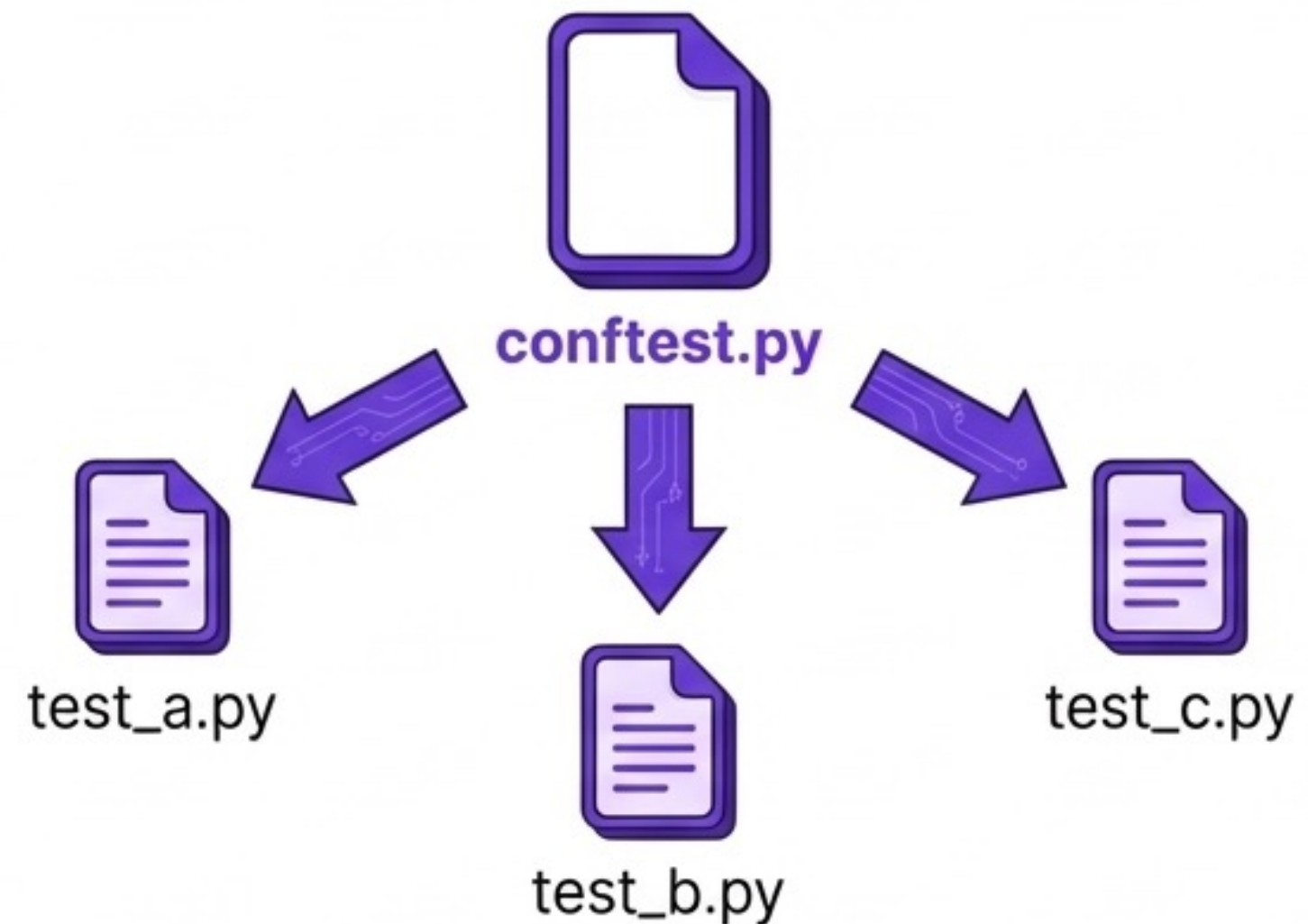
핵심 기능 : Fixture (테스트 환경의 효율적 통제)

테스트 전후의 준비(Setup)와 정리(Teardown) 작업을 모듈화하여 코드 중복을 제거함

Setup : DB 연결, 데이터 세팅

Test : 실제 테스트 실행

Teardown : 자원 해제 및 정리

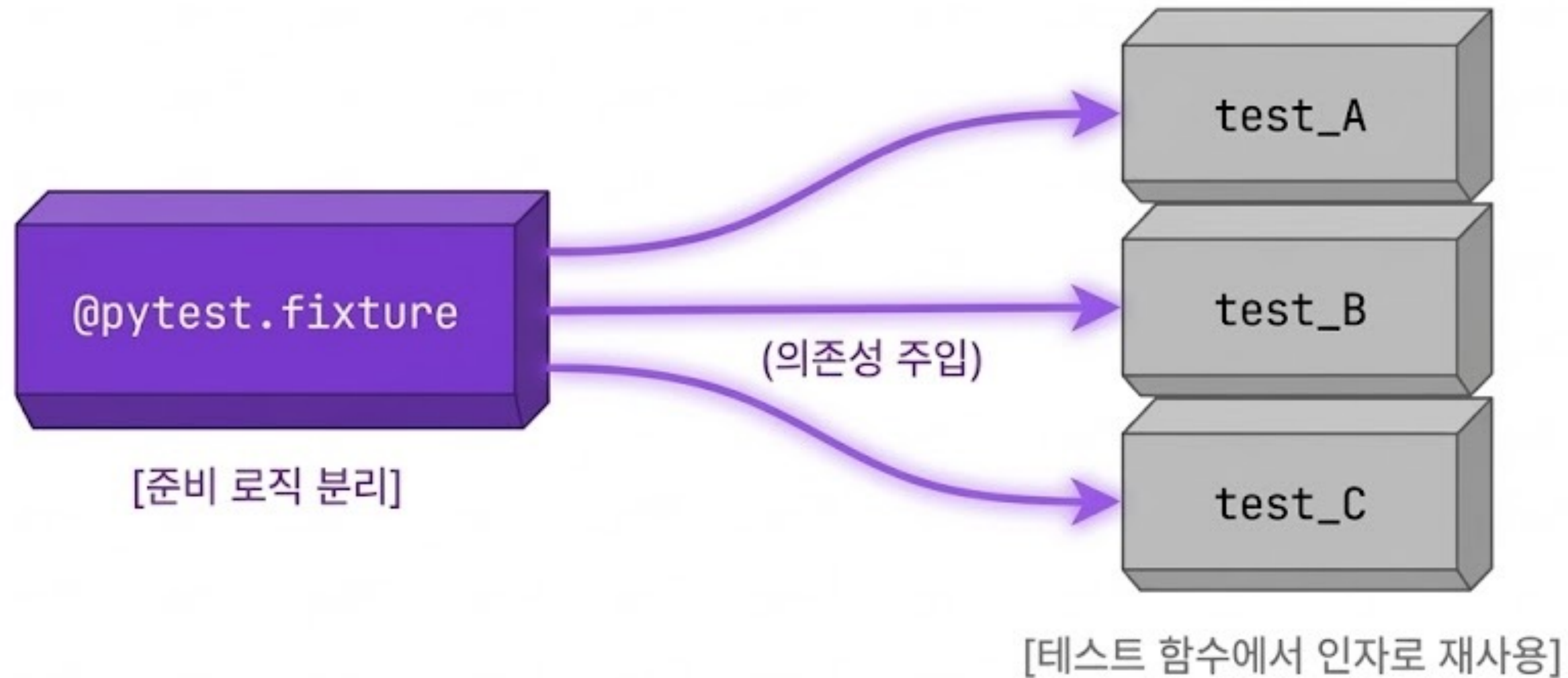


`conftest.py`를 활용하면 여러 테스트 파일에서 Fixture를 중앙 집중식으로 공유할 수 있음



핵심 기능 : Fixture (테스트 환경의 효율적 통제)

테스트 전후의 준비 및 정리 로직을 분리하는 데코레이터





데코레이터(Decorator)란?

```
# 일반 함수
def greet():
    print("Hello")

# 데코레이터 사용
@my_decorator
def greet():
    print("Hello")
```

@데코레이터이름 ← 꾸며줄 도구
def 함수이름(): ← 꾸밈을 받을 함수

함수 호출 시 → 데코레이터의 기능이 함께 실행

핵심 개념

- @ = "이 함수를 꾸며줘"라는 표시
- 함수 자체를 수정하지 않고, 기능을 추가하는 문법
- 같은 부가 기능을 여러 함수에 반복 적용



데코레이터 동작 원리 — 함수를 감싸는 함수

```
# 데코레이터 정의
def my_decorator(func):
    def wrapper():
        print("전: 시작")
        func()
        print("후: 종료")
    return wrapper

@my_decorator
def greet():
    print("Hello")
```

핵심 개념

- 데코레이터는 **함수를 인자로 받아 새 함수를 반환**하는 함수
- 원본을 수정하지 않고 기능을 덧붙임

동작 3단계

- 1 원본 함수를 **인자로 전달**
- 2 **wrapper** 안에서 추가 동작 + 원본 호출
- 3 새 함수(wrapper)를 **반환** → **원본 자리에 대체**



Fixture 문법 : @pytest.fixture

```
# 1. 기본 정의
@pytest.fixture
def sample_user():
    return {"name": "Elice"}

# 2. 테스트 함수에서 매개변수로 받기
def test_name(sample_user):
    assert sample_user["name"] == "Elice"

# 3. scope 옵션 — 재사용 범위 지정
@pytest.fixture(scope="module")
def db(): ...

# function(기본) · class · module · session
```

```
@pytest.fixture(scope=)
def fixture이름():
    return 준비된 데이터
```

인자: 데코레이터 옵션 — scope, params, autouse, ids, name

반환값: fixture로 등록된 함수 (테스트 매개변수로 주입 가능)

조건: pytest 실행 환경 안 (테스트 모듈 / conftest.py)

핵심 포인트

- 일반 함수 위에 **@pytest.fixture** 한 줄
- 테스트 함수의 **매개변수 이름 = fixture 이름**
- return 값이 **자동으로 주입**
- scope로 **재사용 범위** 조절 (function/class/module/session)



conftest.py란? — Fixture 공유 파일

프로젝트 구조

project/

tests/

conftest.py # 공유 fixture

test_user.py

test_order.py

test_payment.py

conftest.py = Fixture 공용 저장소

여러 테스트 파일이 같은 fixture를
사용해야 할 때,

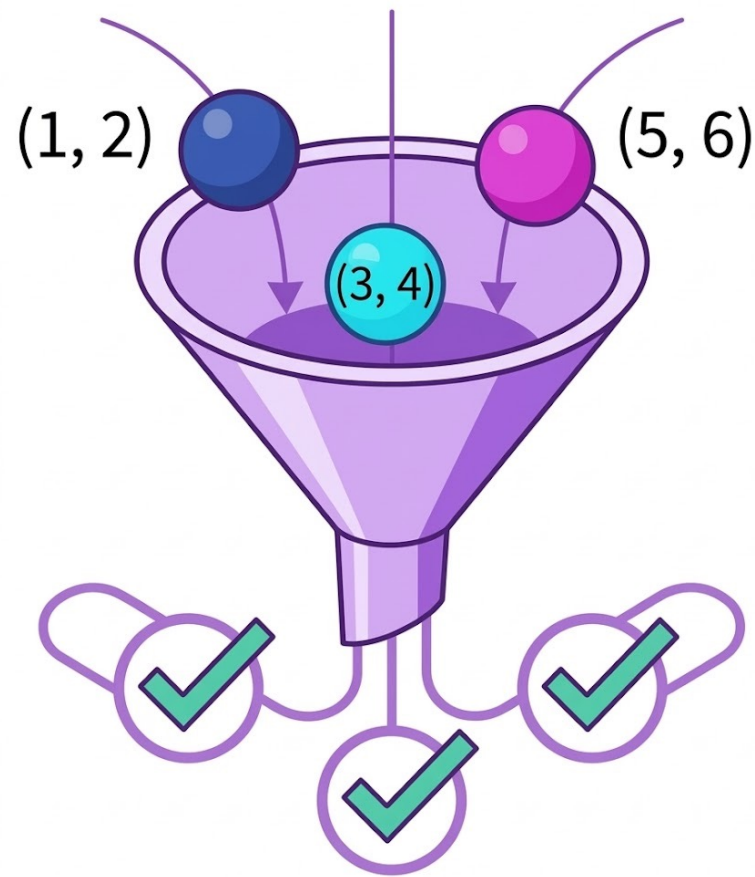
conftest.py에 한 번만 작성하면 끝

핵심 규칙

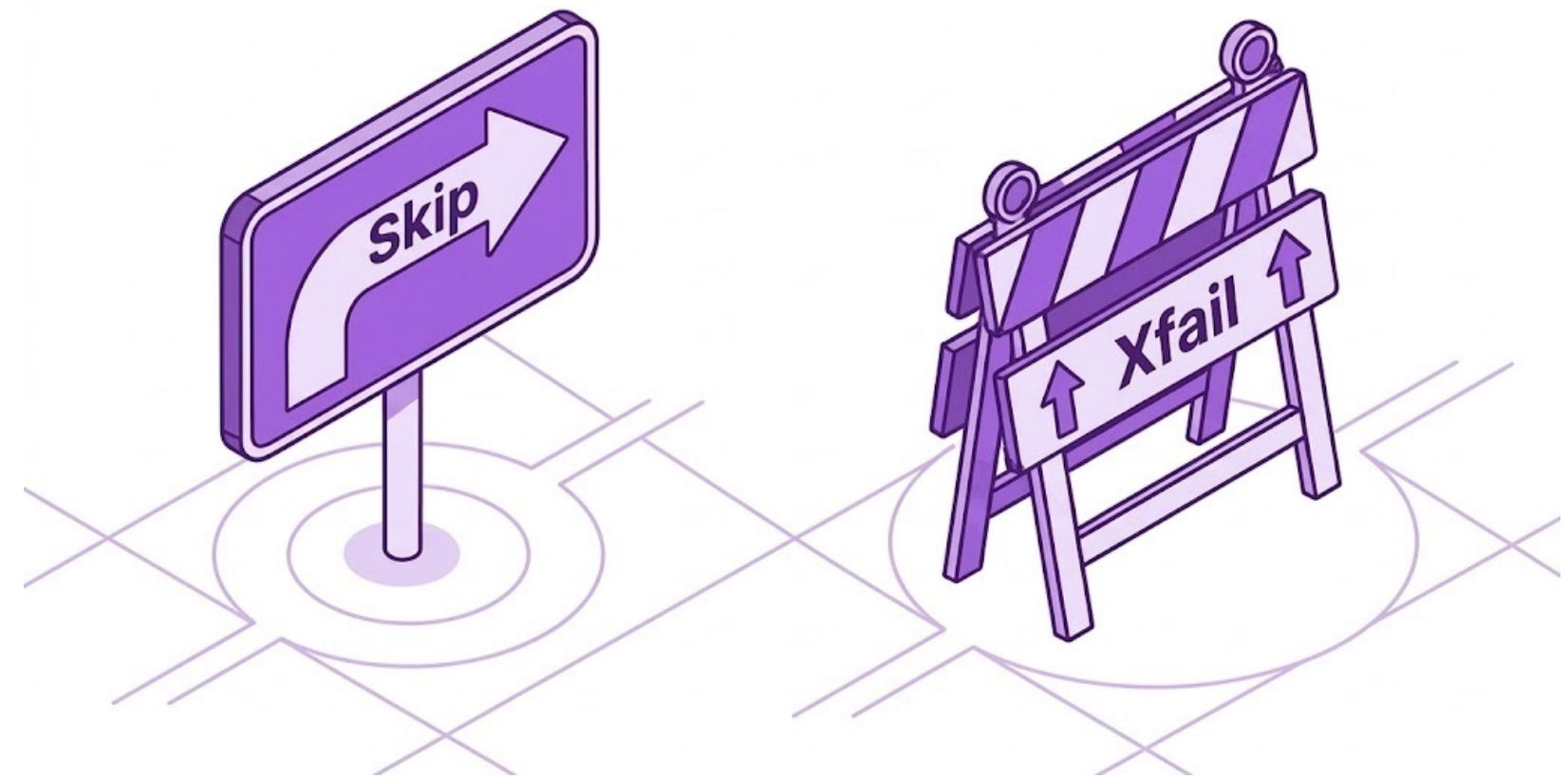
- 파일 이름은 반드시 **conftest.py**
- **import** 없이 자동으로 인식됨
- 같은 폴더 + 하위 폴더에서 사용 가능

핵심 기능 2 : 반복 제어와 예외 처리

`@pytest.mark.parametrize`



하나의 테스트 함수에 여러 입력값을 주입하여 반복 테스트를 간결하게 수행



Skip : 미완성되었거나 특정 환경에서 실행할 수 없는 테스트 건너뛰기

Xfail : 실패가 예상되는 테스트를 명시하여 전체 파이프라인의 붕괴 방지



Marker — 테스트에 라벨 붙이기 : @pytest.mark

```
# 1. 내장 마커 — 건너뛰기
@pytest.mark.skip(reason="WIP")
def test_unfinished(): ...

# 2. 커스텀 마커 — 라벨로 묶기
@pytest.mark.slow
def test_heavy_calc(): ...

# 3. parametrize — 입력값 반복
@pytest.mark.parametrize("x,expected",
                        [(1,2),(3,4)])
def test_inc(x, expected):
    assert x+1 == expected

# 실행 시 라벨로 필터링
$ pytest -m slow
```

@pytest.mark. *마커이름*

대상: 테스트 함수 / 클래스

인자: 마커별 다름 (reason, condition, 데이터셋)

효과: 실행 · 건너뛰기 · 반복 제어

핵심 포인트

- 내장 — skip · xfail · parametrize
- 커스텀 — @pytest.mark.slow 등 자유 정의
- pytest -m 으로 라벨만 골라 실행
- 커스텀은 pytest.ini에 등록 권장



비슷한 테스트, 매번 복사 & 붙여넣기 해야 할까?

```
def test_add_1():  
    assert add(1, 2) == 3
```

```
def test_add_2():  
    assert add(0, 0) == 0
```

```
def test_add_3():  
    assert add(-1, 1) == 0
```

코드가 길어지고 관리가 번거로워짐

단 한 가지 입력만 보는 것보다 여러 입력 조합을 짧고 명확하게 검증해야 하는 경우가 더 많음

데이터 매트릭스 : 로직은 하나로, 데이터는 여러 개로

a	b	expected
1	2	3
0	0	0
-1	1	0
10	5	15

테스트 코드 중복 감소,
다양한 입력 확인,
쉬운 테스트 케이스 추가

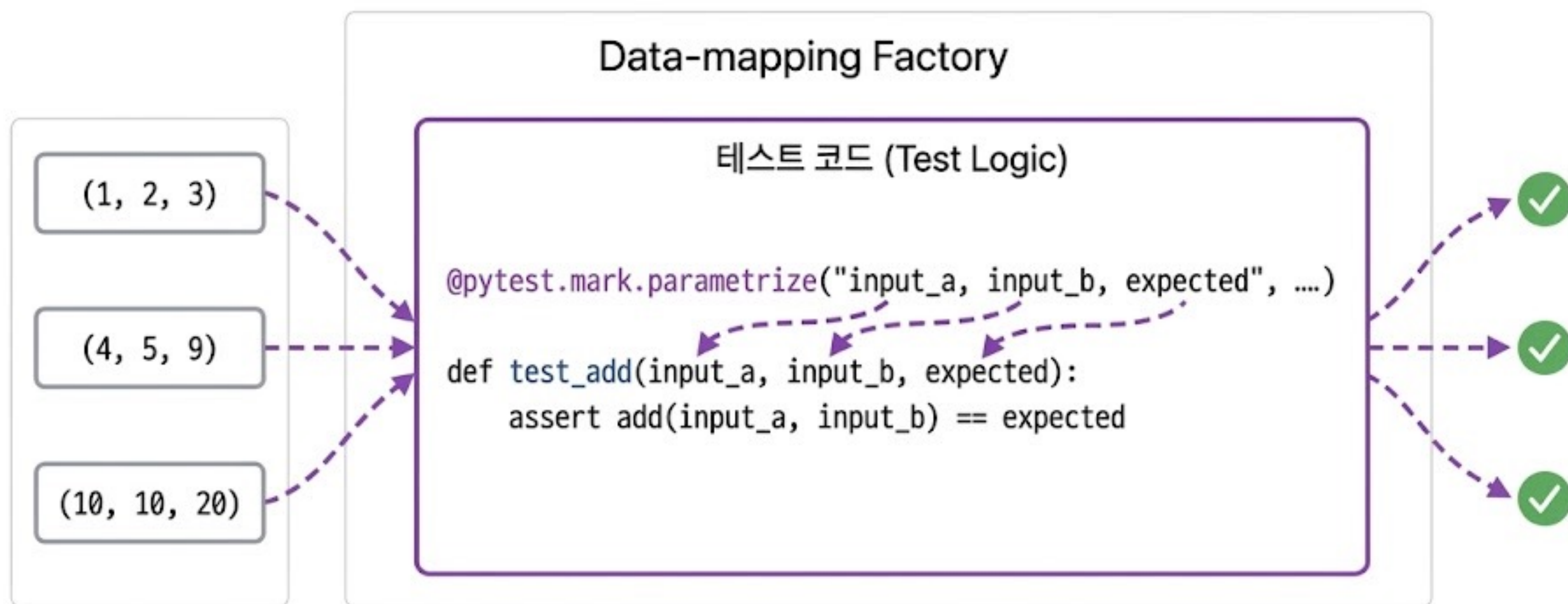
`@pytest.mark.parametrize`

`Test_add_cases(a, b, expected)`



테스트 파라미터화 문법 (@pytest.mark.parametrize)

하나의 테스트 함수에 여러 입력값과 결과 값을 전달하여
중복 코드를 제거하고 케이스를 무한히 확장





파라미터화 코드 적용

```
@pytest.mark.parametrize(  
    "a, b, expected",  
    [  
        (1, 2, 3),  
        (0, 0, 0),  
        (-1, 1, 0),  
        (10, 5, 15),  
    ]  
)  
def test_add_cases(a, b, expected):  
    assert add(a, b) == expected
```

변수명 매핑

각 튜플이 하나의 독립적인
테스트 케이스로 자동 실행됨



파라미터화 : parametrize

기본 형태

```
@pytest.mark.parametrize(
    "인자이름1, 인자이름2",
    [
        (값1, 기대값1),
        (값2, 기대값2),
        ...
    ]
)
def test_함수(인자이름1, 인자이름2):
    ...
```

@pytest.mark.parametrize

- 첫 번째 인자 — 매개변수 이름들을 **심표로 구분된 문자열**로 전달
- 두 번째 인자 — 케이스 리스트. 각 케이스는 **튜플**로 작성하며 **매개변수 순서와 일치**

핵심 포인트

- 케이스 결과는 **test_xxx[95-A]** 형태로 구분
- 경계값 · 예외 케이스 · 수식 검증에 적합
- 케이스 추가 = **튜플 한 줄 추가**로 끝
- 준비 단계가 다르다면 → **별도 함수로 분리**



실행 전략 관리 (markers)



`@pytest.mark.skip`

무조건 건너뛰기

아직 구현되지 않은 기능이나
임시로 제외할 테스트에 사용



`@pytest.mark.skipif`

조건부 건너뛰기

특정 OS 환경이나 라이브러리 버전
등 조건이 맞을 때만 제외



`@pytest.mark.xfail`

실패가 예상되는 테스트

현재 버그가 있거나 실패할 것을 알지만,
추적을 위해 남겨둘 때
(실패해도 전체 결과는 Pass 처리)



마커 사용 예제 (markers)

```
import pytest

@pytest.mark.skip(
    reason="아직 미구현"
)
def test_new_feature():
    result = new_feature()
    assert result == "OK"
```

`@pytest.mark.skip`

항상 건너뛴
실행 결과: **SKIPPED** 표시

```
import sys, pytest

@pytest.mark.skipif(
    sys.platform == "win32",
    reason="Linux 전용"
)
def test_unix_only():
    assert run_shell() == 0
```

`@pytest.mark.skipif`

조건 충족 시 건너뛴
Windows에서만 **SKIPPED**, 그 외엔 정상
실행

```
import pytest

@pytest.mark.xfail(
    reason="버그 #123 추적 중"
)
def test_known_bug():
    assert buggy_func() == 10
```

`@pytest.mark.xfail`

실패해도 통과로 처리
실패 시 **XFAIL**, 통과 시 **XPASS**
strict=True로 주면 **XPASS**가 실패로 처리